Practical 1: Study and Implementation of Advanced Data Structures in Python

Aim

To study and implement advanced data structures such as Stack, Queue, and Linked List in Python.

Theory

A Stack is a linear data structure that follows the LIFO (Last In First Out) principle, where insertion and deletion happen at one end called the top. The two primary operations are push (insert) and pop (remove), both performed in O(1) time; stacks are used in function call management, expression evaluation, undo mechanisms, and backtracking algorithms. A Queue follows the FIFO (First In First Out) principle, where insertion (enqueue) happens at the rear and deletion (dequeue) at the front; unlike a plain Python list, collections.deque is implemented as a doubly linked list internally, giving O(1) time complexity for operations at both ends instead of O(n) for a list. A Linked List is a linear collection of nodes where each node stores data and a reference (pointer) to the next node, allowing dynamic memory allocation and efficient insertion/deletion (O(1) once the position is known) unlike arrays, which require contiguous memory and O(n) shifting. Variants include singly linked lists (one-directional traversal, used here), doubly linked lists (traversal in both directions), and circular linked lists (last node points back to the first). Together, these three structures form the foundation for more advanced data structures such as trees, graphs, and hash tables.

Algorithm

1. Create an empty list to act as a stack.
2. Push elements onto the stack using append() and pop the last element using pop().
3. Create a deque to act as a queue.
4. Enqueue elements using append() and dequeue using popleft().
5. Create a linked list using nested dictionaries with 'data' and 'next' keys.
6. Traverse the linked list from head to tail, collecting values until next is None.
7. Display the results of all three data structures.

Program

```
stack = []
stack.append(10)
stack.append(20)
stack.append(30)
print("Stack:", stack)
print("Popped:", stack.pop())

from collections import deque
queue = deque()
queue.append("A")
queue.append("B")
queue.append("C")
print("Queue:", list(queue))
print("Dequeued:", queue.popleft())

node = {"data": 1, "next": {"data": 2, "next": {"data": 3, "next": None}}}
values = []
cur = node
while cur:
    values.append(cur["data"])
    cur = cur["next"]
print("Linked List:", values)
```

Output

```
Stack: [10, 20, 30]
Popped: 30
Queue: ['A', 'B', 'C']
Dequeued: A
Linked List: [1, 2, 3]
```

Result / Conclusion

The stack, queue, and linked list data structures were successfully implemented and their basic operations (push/pop, enqueue/dequeue, traversal) were verified.

Viva Questions

Q1. What is the difference between a Stack and a Queue?

Ans: A Stack follows LIFO (Last In First Out) — the last element inserted is the first to be removed (like a pile of plates). A Queue follows FIFO (First In First Out) — the first element inserted is the first to be removed (like a ticket line). Stack operations are push/pop at the top; Queue operations are enqueue at the rear and dequeue from the front.

Q2. Why is collections.deque preferred over a list for implementing a Queue in Python?

Ans: list.pop(0) (dequeue from the front) is O(n) because all remaining elements must be shifted one position left. collections.deque is backed by a doubly linked list and provides O(1) popleft() and append(), making it the correct choice for FIFO queues where both ends are accessed.

Q3. What is a node in a Linked List, and what does it contain?

Ans: A node is the basic unit of a linked list. It contains two parts: the data field (the actual value stored) and the next field (a reference/pointer to the next node in the sequence). The last node's next field is None, marking the end of the list.

Q4. What is the time complexity of push and pop operations on a Stack?

Ans: Both push (append to end) and pop (remove from end) on a Python list-based stack are O(1) amortised time, because Python lists use dynamic arrays with amortised constant-time appends and direct index access at the last position.

Q5. What are the real-world applications of a Stack?

Ans: Stacks are used in: (1) function call management — the call stack tracks active function frames; (2) expression evaluation and conversion (infix to postfix); (3) undo/redo mechanisms in text editors; (4) backtracking algorithms (maze solving, DFS); (5) browser history (back button).